

Day - 1

Java Programming

Why do we use Programming Languages ?

Just like two persons use language such as Hindi, English or Marathi to communicate with each other. We use Programming languages to communicate with a computer. A computer mainly understand machine language (0s and 1s), so we use Programming Language such as C, C++, Python and Java to give instruction to the computer in a human-understandable way.

Why do We use Java ?

- Free to use
- Object-Oriented Programming language
- Platform Independent : (Write Once, Run Anywhere)

{	Window
	Linux
	Mac
- Automatic Garbage Collection

Java automatically remove objects that are no longer needed from memory. This process is called Garbage Collection, & it helps manage memory automatically.

- Secure
- JIT (Just-In-Time) Compiler

JIT Compiler converts frequently used Java bytecode into native machine code at runtime, which helps improve the execution speed of Java Programs.

- Widely used.

Draw.io :

Draw.io is a diagramming tool used to visually design and explain software system, workflows, and architectures.

Why Why do We use it ?

- Visualize complex system clearly
- To design system architecture before coding
- To explain the flow and connection between component
- To document project of team member

Day 1: First Programm

File Name: HelloWorld.java

Java Extension use to Write a Java Program

H and W is capital

S is capital

```
Public class HelloWorld {  
    public static void main (String args[])  
    {  
        System.out.println ("Hello World");  
    }  
}
```

Output : Hello World

Steps to execute the Program

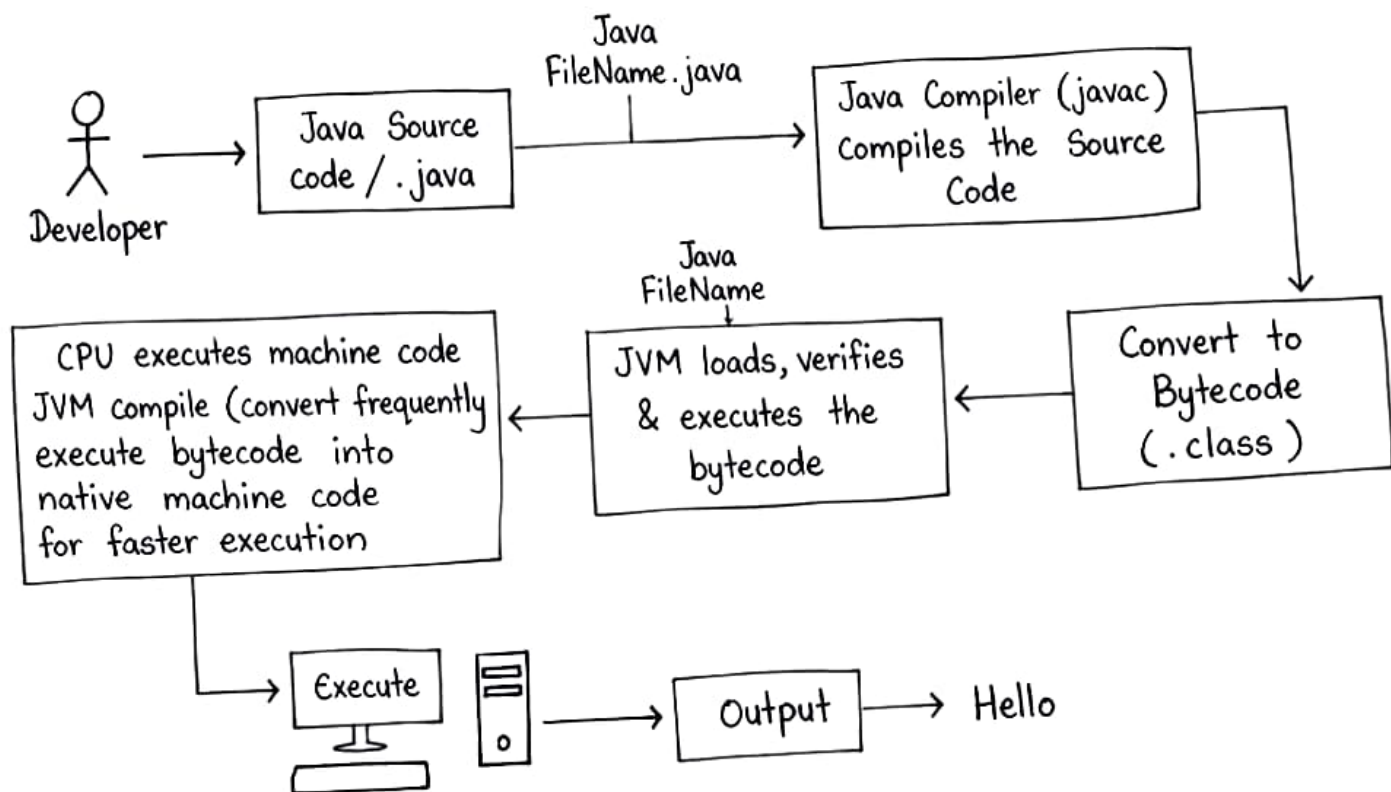
Step 1 : go to cmd & go f into the folder where java file exist

Step 2 : Compile a file using "javac FileName" eg "javac HelloWorld.java"
& after it will create .class file with byte code file with .class
eg : "HelloWorld.class"

Step 3 : execute the byte file using "java Filename" eg "java HelloWorld"
and need to add the class extension

Day-2

Architecture of Java : Task - Print Hello



Rules of Java :

- ① Java is case-sensitive
e.g `int age = 15 ;`
`int Age = 20 ;`
age & Age are different variables
- ② Every statement usually ends with ';' (semicolon)
- ③ Class names usually start with a Capital letter
- ④ Variable & Method names usually start with a lowercase letter
- ⑤ The program start from the `main()` method
- ⑥ Java uses `{ }` to define blocks of code
- ⑦ Variable need a data type
- ⑧ Java keyword cannot be used as variable names
e.g `int class = 10 ;` // throw error, class is a keyword
- ⑨ String are written inside double quotes
- ⑩ Comment are ignored by Java
- ⑪ Java mainly follows case sensitivity + semicolon + data type + `{ }`
(Proper naming + `main()`)

Example: Welcome Java

Day-2 (Greeting)

```
public class Greeting {  
    public static void main(String args[]) {  
        System.out.println("Hello, Welcome in Day-2 of Java Coding");  
        System.out.println("Hey, this is Greeting class");  
    }  
}
```

Details & Rules

① class name & File name always be the same of if we don't want the file name is similar to class then we don't need to add 'public' keyword in class See in above example. it will works

② only one Public class in one single file
e.g.

```
public class Greeting {  
    }  
class Session {  
    }
```

③ Cannot have two public classes in one file
e.g.

```
public class Main {  
    }  
public class Student {  
    }
```

Code :

Welcome.java → File Name.java

```
class Greeting {  
    public static void main (String args[]) {  
        System.out.println ("Greeting class");  
    }  
}
```

```
class Session  
{  
    public static void main (String args[]) {  
        System.out.print ("Session class");  
    }  
}
```

Execution

What will happen when we execute the Welcome.java file ?

See :

- First compile javac Welcome.java
- then it will create two class files

├ Greeting.class
└ Session.class

- if we execute "java Greeting" then it will only run the Program of Greeting class & Provide output as "Greeting class"
- if we execute "java Session" then it will only run the Program of Session class & Provide output as "Session class"

* What happens if we don't write any main method in class ?

- then Java will not get any entry point into the class, so it produce the error that is "main method not found".

Day 3

8. SBI Account → class in SBI net banking will represent Customer account info

Code : Display account information → AccountInfo

```
class AccountInfo
{
    public static void main (String a[])
    {
        System.out.println ( " Bank Name : SBI " );
        System.out.println ( " Branch : Nandurbar " );
        System.out.println ( " Account Holder : Sudarshan Patil " );
        System.out.println ( " Ammount : 0 " );
        System.out.println ( " Type : saving " );
    }
}
```

Static vs Dynamic Languages

Static

- Perform type checking at Compile time
- Errors will show at compile time
- Declare datatype before you use it
- More Controls

e.g Java

```
int age = 20 ;
```

if (i write age = "hello" as age typed integer in java compiler gives error



```
age = "hello" // Compile time error  
//Error: String cannot be converted to int
```

C++

```
int age = 20 ;
```

```
age = "hello" ; // Type-error
```

Dynamic

- Perform type checking at runtime
- Error might not show till program is run
- No need to declare datatype of variable
- Saves time in writing code but might give error at runtime

Python

```
x = 10
```

```
Print(x) # 10
```

```
x = "hello"
```

```
Print(x) # hello
```

JavaScript

```
let x = 10 ;
```

```
console.log(x) // 10
```

```
x = "hello" ;
```

```
console.log(x) // hello
```

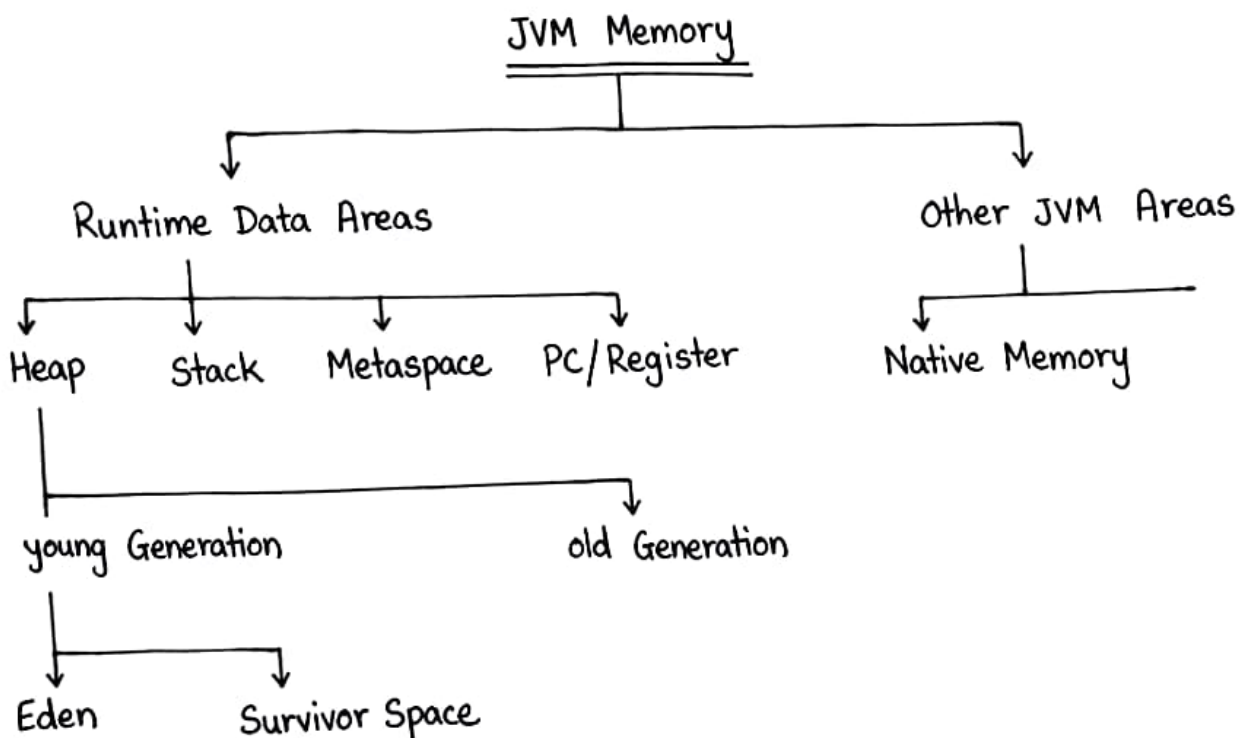
Java Memory Management

Java memory management is primarily handled by the JVM (Java Virtual Machine) and Garbage Collector (GC)

★ Important interview concept

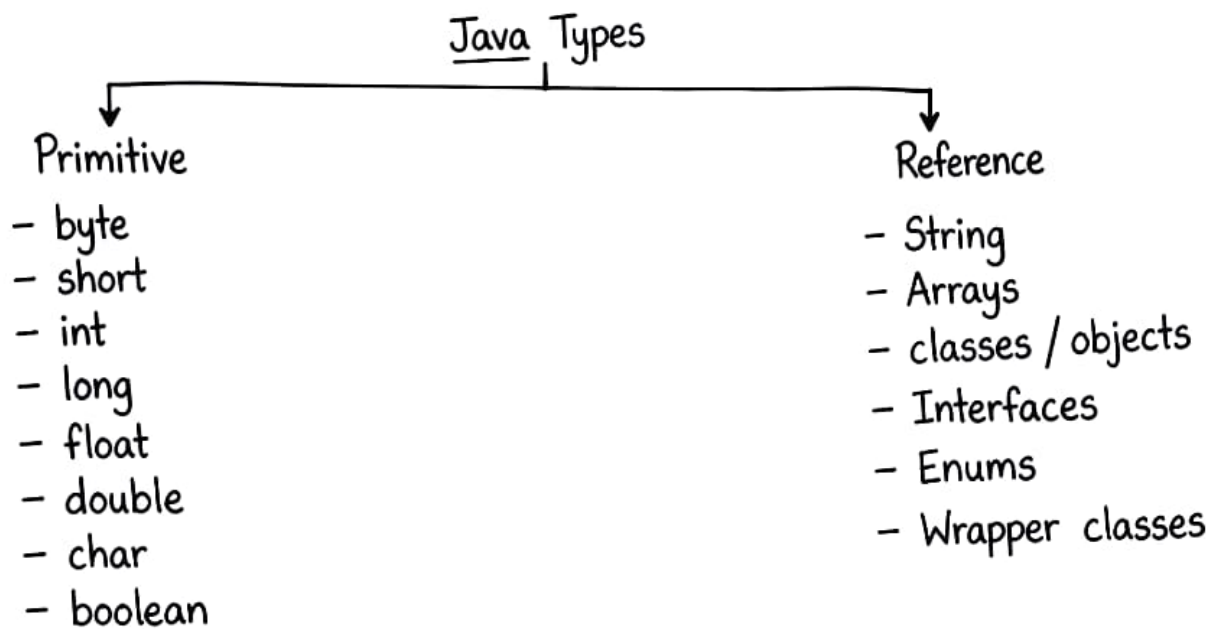
- Primitive variable stores value
- Reference variable store reference to objects

Jvm Memory Architecture



This is a simplified model. The JVM specification does not mandate that every local variable must physically reside on the stack or every object must physically reside in the heap. JVM implementations and JIT optimizations can change the actual representation.

Primitive vs Reference Type

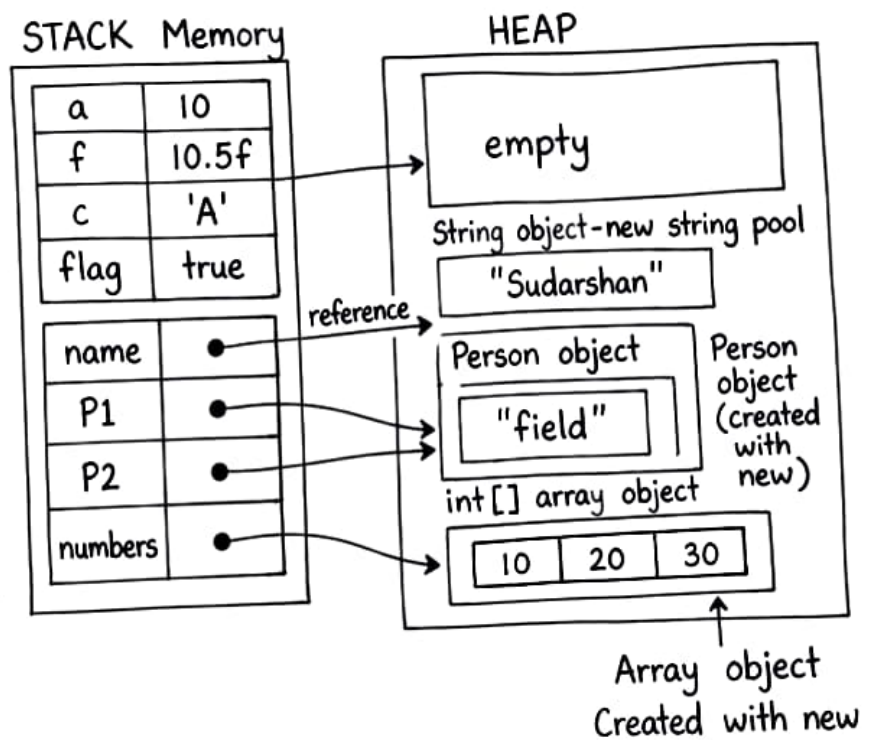


Java Memory (Stack vs Heap)

- Primitive types (int, float, char, boolean) Store the actual value in stack.
- Non-Primitive types (String, object, arrays) Store reference in the stack, and the actual object is created in the heap
- Multiple reference (P1 & P2) can point to the same object
- Each new object (new Person(), new inting[]) is stored in the heap
- String literals (e.g "Sudarshan") are stored in the String Pool (apart of Heap)

// Example Code

```
int a = 10 ;  
float f = 10.5f ;  
char c = 'A' ;  
boolean flag = true ;  
  
String name = "Sudarshan"  
Person P1 = new Person();  
Person P2 = P1 ;  
int[] numbers = {10,20,30}
```



Command Line Arguments

Command-Line arguments are values passed to a Java Program when you run it from the terminal or cmd.

e.g

Public class Demo

{

public static void main (String args[]) {

↓
args is a Array of String

args

0	1	2	3	4
---	---	---	---	---

Suppose I run the "Java Demo Rahul 101 IT"

↓
in Java, This command-line argument are recieved throug String[] args

args

Rahul	101	IT
0	1	2

args[0] ⇒ Rahul ⇒ String

args[1] ⇒ 101 ⇒ String

args[2] ⇒ IT ⇒ String

It means the args array can contain only String .

e.g. java Demo Rahul 101 25.5 true

args[0] ⇒ "Rahul" ⇒ String

args[1] ⇒ "101" ⇒ String

args[2] ⇒ "25.5" ⇒ String (Even)

args[3] ⇒ "true" ⇒ String

- If later we want 101 as an actual integer, then we convert using `parseInt()`

e.g
`int id = Integer.parseInt(args[1]);`

Convert String → Float

`float price = Float.parseFloat(args[0]);`

Convert String → Boolean

`boolean status = Boolean.parseBoolean(args[0]);`

e.g Source Code

`class ConvertingType`

`{`

`public static void (String args[])`

`{`

`int age = Integer.parseInt(args[0]);`

`float height = Float.parseFloat(args[1]);`

`boolean student = Boolean.parseBoolean(args[2]);`

`System.out.println(age);`

`System.out.println(height);`

`System.out.println(student);`

`}`

`}`

String args:

"27"	"5.4"	"true"
------	-------	--------

Step1 : Compile Source code ⇒ `javac ConvertingType.java`

Step2 : Execute File ⇒ `java ConvertingType`

27	5.4	true
----	-----	------

Class AccountInfo

```
{  
    public static void main(String args[])  
    {  
        System.out.println("This is account info...");  
    }  
}
```

Notes:

① class AccountInfo {

}

- class is keyword used to define a class
- AccountInfo is class name.

② Public class AccountInfo

{

}

- Public is an access modifier.
- It means the class or members can be accessed from other classes, subject to Java's access rules.

Public Static void main(String args[])

Public → JVM can access the main() method from outside the class

③ Static :- Static means the method belongs to class itself, rather than to an object of that class.

- main() is static, the JVM can invoke it without first creating an object of the class.

e.g. class Test {
 static void show() {
 System.out.println("Hello");
 }
 public static void main(String[] args) {
 Test.show(); // no object required
 }
}

with the help of static
keyword
↓

Note : Here JVM calls the class without creating an object, that's why main() is static eg AccountInfo.main()

Without static, an object would normally be required

```
class Test {  
    void show() {  
        System.out.println("Hello");  
    }  
    public static void main(String[] args) {  
        {  
            Test obj = new Test();  
            obj.show();  
        }  
    }  
}
```

① main()

Public static void main(String [] args)

↓ method

- `main()` is a standard entry point for Java application.
- JVM looks for a compatible main method and starts execution there.
- `main` is the method name.

String args []

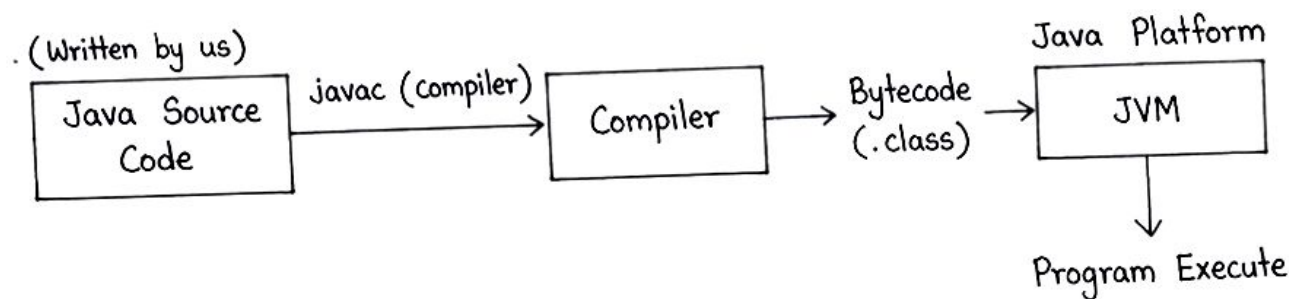
- ② It is a String Array.
- ② It is used to receive Command-line arguments.

Uses / Advantages

- Provide → Accesible to the JVM.
- Static. → Can be called without. craating an object.
- Void → returns 'no vave'.
- main() → Central place to write the starting code.
- String args[] → Command-line arguments.

JVM (Java Virtual Machine)

The Java is responsible for executing Java bytecode.



- JVM also handles things such as :

- Loading classes
- Runtime memory management
- Bytecode execution
- Garbage collection (GC)
- Runtime checks

- JVM is platform-independent, but Java bytecode is designed to achieve platform independence.

JRE (Java Runtime Environment)

JRE provides the runtime environment and predefined Java class libraries that we use while running a Java program.

e.g.

```
String name = "Sudershan";  
System.out.println(name);  
int number = Integer.parseInt("100");
```

Flow :

- String → Predefined Java class
- System.out → Predefined Java class
- println() → Predefined method (Prints message on console/terminal)
- Integer → Predefined Java class
- parseInt() → Predefined method used to convert String into an integer

- We do not to write these classes and method from scratch because Java provides them through its standard libraries.

Example

Suppose we want to print something

```
System.out.println("Hello");
```

- We don't write the System class or println() method because Java already provide them.

Similar

```
int x = Integer.parseInt("10");
```

- Integer → class
- parseInt → method name
- Integer.parseInt("10"), || converting string to Integer

Note - JRE gives us the runtime environment along with Java's predefined libraries/classes, so we can directly use things like string, System, Integer println(), parseInt(), etc, insted of creating everything from scratch.

If Java did not provide these standard classes/methods, we would have to implement that functionality ourselves.

JDK

- The JDK is used to develop Java application

Tools

- Javac → Java Compiler
- Java → launches Java application
- Jar → create / manages JAR files
- Javadoc → generates documentation

JDK

- Development tools
- JVM
- Java runtime libraries

Compiler & Decompiler

Compiler

Java



.class

Javac



.class

e.g

Javac AccountInfo.java

This produce,
AccountInfo.class

Decompiler

- Decompiler we use sometime we lost our source code. then with the help of .class file we can generate source code.

.class



Decompiler



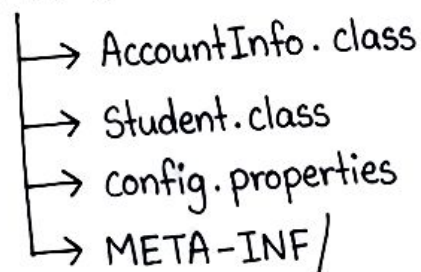
Java-like source code.

JAR

- Jar stands for Java Archive
- A jar file is a package feature used to bundle Java classes and other resources into single file
- It is used on the ZIP archive format
- A jar can contain,
 - class files
 - configuration files
 - images
 - metadata
 - other resources

e.g

myapp.jar



Extracting a JAR.

jar -xvf file.jar
↓
fileName

Java Data Types

What is Data Type ?

A data type tells java what type of value a variable can store

For e.g

```
int age = 20 ;
```

- . int → Data Type
- . age → Variable Name
- . 20 → Value

int → tells java that age variable is going to store a whole number or integer number.

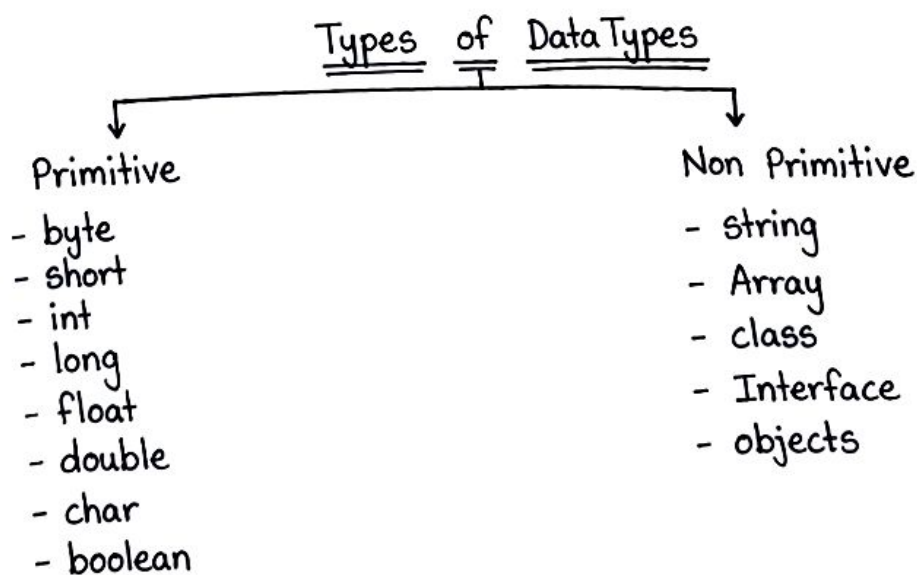
e.g2

```
double price = 99.50 ;
```

- . double → Data Type
- . Price → Variable Name
- . 99.50 → Value

So double tells java that the variable price can store decimal number

Simple :- Data Type mean the type of data that a variable can store



Java Data Type - Range Chart

<u>Data Type</u>	<u>Bits</u>	<u>Bytes</u>	<u>Range Values</u>
- byte	8 bits	1 byte	- 128 to 127
- short	16 bits	2 byte	- 32,768 to 32,767
- int	32 bits	4 byte	- 2^{31} to $2^{31}-1$
- long	64 bits	8 byte	- 2^{63} to $2^{63}-1$
- float	32 bits	4 byte	
- double	64 bits	8 byte	
- char	16 bits	2 byte	0 to 65,535
- boolean	Not fixed by java	JVM-dependent	true/false

Class Variable and Local Variable

Class Variable - If a variable is declared inside a class but outside method ; it is called a class variable

- If it has static ; it is called a static variable

Local Variable - If a variable is declared inside a class but outside method ; it is called a local variable

e.g

```
class DataType
{
    static int number ; // class variable | static variable
    public static void main (String args[])
    {
        System.out.println(number) ; // 0
    }
}
```

Default Values

Java automatically gives the class variable its default value

Integer data Types Default Value.

<u>Data Type</u>	<u>Default Value</u>
- byte	0
- short	0
- int	0
- long	0

Decimal data Type

<u>Data Type</u>	<u>Default Values</u>
- float	0.0
- double	0.0

Bool

Boolean

boolean → Default Value → false

Char

"\u0000"

The "char" default value normally looks blank when we print it.

e.g

static char ch;

System.out.println("char is - " + ch);

Output

char is -
 ↑ blank

Local Variable

A local Variable is a Variable declared inside a method , constructor or block

e.g

```
Public static void main(String args[])  
{  
    Short number = 2000 ;  
    System.out.println(number) ;  
}
```

output

2000

Important Point

For local Variables , Java does not gives a default Value

e.g

```
Public static void main(String args[]) {  
    Short number ;  
    System.out.println(number) ;  
}
```

This gives an error:

Variable number might not have been initialized

So, we must assign a value before using a local Variable

Correct

```
Short number = 2000 ;  
System.out.println(number) ;
```

- Simple Difference of class / static & local Variable

e.g class / static Variable

```
class DataType
{
    static int number;
    public static void main(string args[])
    {
        System.out.print(number);
    }
}
```

output

0

- Java gives the default value automatically

Local Variable

```
Public class DataType
{
    public static void main(string args[])
    {
        int number;
        System.out.print(number);
    }
}
```

Error :

- Variable number might not have been initialized
- Java does not gives a default Values to local Variable.

Wrapper classes

Java also provides classes for primitive data types

for e.g

float → Float
int → Integer
byte → Byte
short → Short
long → Long
double → Double
char → character
boolean → Boolean

e.g

```
Float num = 667.0f;
System.out.println(num);

Integer num1 = 45;
System.out.print(num1);

Integer → class
int → primitive Data Type
```

Code

```
Public class DataType
```

```
{
    static short anmount ;
    static byte num1 ;
    static int num2 ;
    static long num3 ;

    // Decimal class Datatypes
    static float price ;
    static double Price2 ;
    static boolean status ;
    static char ch ;
```

```
Public static Void main(String args[])
```

```
{
    // Local Variable
    Short num = 2000 ;
    System.out.print(num) ;
```

*

Local Variable -

if we create a local Variable and do not assign any Value, Java will not give a default Value.

e.g

```
Short number ;
System.out.print(number) ;
```

Error:

Variable number might not have been initialized

*/

// Printing class (static Variables

```
System.out.println("Anmount is : " + anmount) ; // default Value → 0
System.out.println(" num1 : " + num1) ; // = 0
System.out.println(" num2 : " + num2) ; // = 0
System.out.println(" num3 : " + num3) ; // = 0
System.out.println(" Price : " + price) ; // default Value → 0.0
System.out.println(" Price2 : " + Price2) ; // default Value → 0.0
System.out.println(" status : " + status) ; // default Value → false
System.out.println(" char : " + ch) ; // default Value → blank
```

```
}
```

```
}
```

Wrapper Class

A Wrapper class is a class that is used to convert a primitive data type into an object

Java has wrapper classes for all 8 primitive data types.

<u>Primitive</u>		<u>Wrapper Class</u>
int	→	Integer
byte	→	Byte
short	→	Short
long	→	Long
float	→	Float
double	→	Double
char	→	Character
boolean	→	Boolean

e.g

```
public class WrapperClass
{
    public static void main(String args[])
    {
        // primitive datatype
        int num = 45;
        System.out.println(num); // 45
        int num1 = 45 + 56;
        System.out.println(num1); // 45+56
        char ch = 'A';
        System.out.println(ch); // A

        // class / static boolean
        Integer number = 45;
        System.out.println(number);
        Boolean status = false;
        System.out.println(status);
        Character ch = 'A';
        System.out.println(ch);
    }
}
```

output

45
45+56
A
45
false
A

Important Note

Integer num1 = 45

Java automatically converts the primitive int value 45 into an Integer object. This is called autoboxing

int num = num1;

automatically converts the Integer object back to int. This is called unboxing

Common Errors in Java (Compile & RunTime)

Remember

Compile Time error → Caught before Running

Runtime error → occur while Running

- ① Error : Syntax error / Illegal expression

Example :
- int x = ;
- if (x > 5 {
- for (int i=0; i<5; i++ {

Compile message : illegal start of expression / ';' expected ')' ; expected

- ② Error : Missing Semicolon

Example : int x = 10
System.out.print(x)

Compiler message : ';' expected

- ③ Error : Undeclared Variable

Example : System.out.print(x);

Compiler message : cannot find symbol

- ④ Error : Incompatible types

Example : int x = "Hello";
boolean b = 10 ;

Compiler message : incompatible type String cannot be converted to int

- ⑤ Error : Variable might not have been initialized

Example : int x ;
System.out.print(x);

Compiler message : Variable x might not have been initialized.

⑥ Error : Duplicate Variable

Example : `int x = 10 ;`
`int x = 20 ;`

Compiler message : Variable x is already defined

⑦ Error : Duplicate class

Example : Two classes with same name

```
class Demo {  
    }  
class Demo {  
    }
```

Compiler message : duplicate class

⑧ Error : Bad operand types

Example : `boolean a = true ;`
`boolean b = false ;`
`System.out.print(a+b) ;`

Compiler message : bad operand types for binary operator '+'

⑨ Error : Access Violation

Example : Accessing a private variable from outside it class

```
class A {  
    private int x = 10 ;  
}  
class B {  
    public static void main(String[] args) {  
        A obj = new A();  
        System.out.print(obj.x);  
    }  
}
```



Compiler message : has private access

⑩ Error: Missing class/import

Example: Scanner sc = new Scanner(System.in);
(without import)

Compiler message: Cannot find symbol

⑪ Error: Array index out of bounds: /Wrong Command-line argument index.

Example:

```
class Demo {  
    main(String args[]) {  
        int string name = args[0];  
        String product = args[1];  
        System.out.println(args[3]); // this index not passed.  
    }  
}
```

Java Demo "Sudarshan" "Asus Laptop"

Runtime error

Example message Array Index Bound Exception.

⑫ Error: Invalid number conversion

Example: int x = Integer.parseInt("abc");

Runtime error: Number Format Exception

⑬ Error: Divide by zero

Example: int x = 10/0;

Runtime error: ArithmeticException: / by zero.

⑭ Error: Infinite loop

Example: while(true) { }

(No Compile error / no Runtime error
program keeps running)

(15) Error: break outside loop/switch

Example: break ;

Compiler message: break outside switch or loop.

(16) Error: Continue outside loop/switch

Example: ~~to~~ Continue ;

Compiler message: continue outside switch or loop.

(17) Error: Invalid Condition type

Example: if (10) { 3

Compiler message: incompatible type: int cannot be converted to boolean.

(18) Error: Invalid assignment

Example: 10 = x

Error Message: unexpected type

(19) Error: unclosed String or character literal / too many characters in char

Example: System.out.print ("Hello'");

- char ch = 'A';

- char ch = 'AB'

Error Message: unclosed ~~ch~~ String literal

unclosed character literal

(20) Error: Invalid Identifier / Keyword

Example: int 2num = 10 ;

int class = 10 ;

Error message: identifier expected / not a statement ,